대규모 언어 모델 플랫폼의 엔터프라이즈 아키텍처 벤치마킹 및 **OpenMake LLM** 기능 고도화 전략 보고서

차세대 기업용 **AI** 플랫폼의 아키텍처 패러다임 변화와 벤치마킹의 당위성

현대의 대규모 언어 모델(LLM) 생태계는 단순한 챗봇 형태의 대화형 인터페이스를 넘어서, 기업의 방대한 비정형 데이터를 안전하게 색인하고, 레거시 인증 인프라와 결합하며, 다중 모달리티(Multi-modality) 기반의 복합 추론을 수행하는 '엔터프라이즈 AI 운영 체제'로 진화하고 있다. 이러한 패러다임의 전환 속에서 오픈소스 진영을 선도하고 있는 Open WebUI는 Python 3.11과 FastAPI를 기반으로 "모든 모델, 모든 데이터베이스, 모든 도구"를 연결한다는 범용적 철학을 완벽하게 구현하며 125,000개 이상의 GitHub Star를 확보하는 등 사실상의 업계 표준으로 자리매김하였다.[1] 반면, Node.js와 Express 프레임워크를 기반으로 구축된 OpenMake LLM은 "하나의 모델 백엔드와 깊은 파이프라인"이라는 독자적인 철학 아래, A2A(Agent-to-Agent) 토론 엔진, 심층 연구(Deep Research) 파이프라인, 그리고 2계층 시맨틱 캐시(Semantic Cache)와 같은 고도의 자율형 에이전트 기능을 선도적으로 구현하며 그 잠재력을 입증하고 있다.

그러나 플랫폼 아키텍처 및 기술 스택 비교 분석 결과에 따르면, OpenMake LLM은 에이전트 기반의 추론 파이프라인 영역에서는 독보적인 우위를 점하고 있음에도 불구하고, 엔터프라이즈 환경 도입의 필수 관문인 보안 인증 체계, 이기종 데이터 소스를 아우르는 범용적 정보 검색(RAG) 프레임워크, 지능형 문서 전처리, 그리고 사용자 경험(UX) 및 호환성 측면에서 Open WebUI 대비 중대한 아키텍처적 결함을 내포하고 있다. 특히 폐쇄망(Air-Gapped) 환경에서의 데이터 거버넌스, 역할 기반 접근 제어(RBAC)의 세분화, 하이브리드 검색 기반의 벡터 리트리벌(Retrieval) 생태계는 기업 고객의 채택 여부를 결정짓는 핵심 지표로 작용한다.

본 보고서는 제공된 아키텍처 명세서, 기술 스택 비교, RAG 및 검색 전략, 문서 처리 엔진, 보안 및 인증 구조 등 전방위적인 기술 데이터를 종합하여, Open WebUI의 성숙도를 기준으로 OpenMake LLM이 기업용 AI 플랫폼 시장에서 경쟁 우위를 확보하기 위해 반드시 보완해야 할 요소들을 심층적으로 분석한다. 기능의 비즈니스 임팩트, 데이터 무결성 보장 수준, 그리고 Node.js 생태계 내에서의 기술적 구현 난이도를 종합적으로 평가하여, 보안 및 인프라 통합(1순위), 하이브리드 정보 검색 시스템(1순위), 다중 모드 문서 전처리(2순위), UI/UX 및 시스템 확장성(3순위)으로 카테고리를 분류하고 각 영역별 구체적인 기술 명세와 엔지니어링 구현 전략을 제시한다.

[우선순위 **1]** 엔터프라이즈 보안**,** 글로벌 인증 체계 및 데이터 거버넌스 아키텍처 고도화

대기업 및 공공 기관 환경에서 AI 플랫폼의 성공적인 안착을 좌우하는 가장 결정적인 요소는 기존 사내 아이덴티티 인프라(Identity Provider, IdP)와의 매끄러운 통합 및 무결점의 데이터 거버넌스 확립이다. Open WebUI는 SCIM 2.0 프로토콜, LDAP/Active Directory 통합, 엔터프라이즈급 역할 기반 접근 제어(RBAC) 및 보안 미들웨어를 선도적으로 도입하여 이 분야의 표준을 제시하고 있다.[3] 반면, OpenMake LLM은 단순한 JWT(JSON Web Token) 및 세션 기반의 인증과 범위(Scope) 기반의 제한적인 권한 제어에 머물러 있어 전사적 규모의 확장에 치명적인 한계를 노출하고 있다. 따라서 다음과 같은 아이덴티티 및 보안 아키텍처의 전면적인 개편이 요구된다.

SCIM 2.0 기반 자동화된 프로비저닝 프로토콜 도입 **(Node.js** 아키텍처 구현**)**

조직의 규모가 수천 명 단위로 확장될 경우, 입사, 부서 이동, 퇴사 등 사용자의 수명 주기 이벤트를 관리자가 수동으로 처리하는 것은 불가능에 가깝다. SCIM(System for Cross-domain Identity Management) 2.0은 Okta, Azure AD, Google Workspace와 같은 외부 ID 공급자에서 생성된 사용자 및 그룹의 변동 내역을 AI 플랫폼의 계정 정보와 실시간으로 동기화하는 핵심 표준 프로토콜이다.[3] Open WebUI는 SCIM_ENABLED, SCIM_TOKEN, SCIM_AUTH_PROVIDER 환경 변수를 통해 SCIM 엔드포인트를 제어하며, 외부 ID 연동 시 500 내부 서버 오류를 방지하기 위한 견고한 예외 처리 로직을 갖추고 있다.[3]

OpenMake LLM은 Express.js 라우팅 시스템을 기반으로 RFC 7643(스키마) 및 RFC 7644(프로토콜)를 엄격히 준수하는 RESTful API를 즉각적으로 구축해야 한다.[5] 엔터프라이즈 호환성을 위해 반드시 구현해야 할 핵심 엔드포인트와 동작 명세는 다음과 같다. 첫째, GET /scim/v2/Users 엔드포인트는 userName 및 emails 속성에 대한 조건부 필터링(eq, co, and, or 연산자)과 페이지네이션(startIndex, count) 기능을 지원해야 한다.[3] 둘째, 새로운 사용자를 생성하는 POST /scim/v2/Users 요청 시, 시스템은 urn:ietf:params:scim:schemas:core:2.0:User 스키마 규격을 검증하고 생성된 고유 id 및 meta 정보와 함께 201 Created 상태 코드를 반환해야 한다.[5] 셋째, 조직 개편에 따른 부분 업데이트를 처리하는 PATCH /scim/v2/Users/{id} 엔드포인트는 배열 기반의 작업(add, remove, replace)을 해석하여 다중 값 속성 및 중첩 경로(예: name.givenName)를 무결하게 갱신하는 복잡한 로직이 요구된다.[5] 마지막으로 사용자의 퇴사 시 물리적 삭제(Hard delete) 또는 비활성화(Soft delete, active: false)를 처리하는 DELETE 엔드포인트가 연동되어야 한다.[3] 이러한 모든 통신은 HTTP Bearer 토큰 검증 미들웨어를 통해 보호되어야 하며, 다중 테넌트(Multi-tenant) 아키텍처를 고려하여 요청 헤더를 분석해 적절한 테넌트 컨텍스트로 라우팅하는 기술적 기반이 선행되어야 한다.[5]

레거시 인프라 통합을 위한 **LDAP** 및 **Active Directory** 바인딩 체계 구축

클라우드 네이티브 환경 외에도, 금융권이나 공공기관 등 보안이 격리된 온프레미스 환경에서는 여전히 Active Directory(AD) 또는 OpenLDAP이 주력 인증 시스템으로 활용된다.[3] Open WebUI는 데이터베이스 초기화 및 UI 환경 설정을 통해 서버 호스트, 포트(일반 389, LDAPS 636), 검색 기반(Search Base)을 동적으로 매핑하는 기능을 완비하고 있다.[3] OpenMake LLM은 Node.js 생태계의 ldapjs 또는 activedirectory 모듈을 통합하여 2단계 인증 바인딩 아키텍처를

구현해야 한다.[8]

익명 검색이 엄격히 차단된 엔터프라이즈 환경의 특성을 고려하여, 시스템은 사전에 설정된 서비스 관리자 계정(Bind DN 및 Credentials)을 통해 디렉터리에 1차적으로 바인딩한 후, 사용자가 입력한 식별자를 기반으로 트리를 검색(Search)하여 고유 사용자 DN을 추출하고, 해당 DN과 비밀번호로 2차 바인딩을 시도하여 인증을 완료하는 프로세스를 거쳐야 한다.[3] 아울러 조직마다 상이한 로그인 식별자 속성—예를 들어 Active Directory의 sAMAccountName 또는 userPrincipalName, OpenLDAP의 uid—을 시스템 환경 변수 수준에서 유연하게 매핑할 수 있는 구조가 필요하며 [3], 네트워크 패킷 스니핑을 통한 자격 증명 유출을 방지하기 위해 LDAPS 및 StartTLS 암호화 통신을 강제하는 보안 옵션이 백엔드 미들웨어에 추가되어야 한다.

가산적 논리 기반의 세분화된 접근 제어(RBAC) 및 서버 측 보안 강화

대화형 AI 플랫폼에서 가장 민감한 부분은 지식 베이스(Knowledge Base)에 색인된 기업 데이터의 유출 위험이다. 기술 비교 자료를 분석하면, Open WebUI는 단순한 권한 부여를 넘어 AccessGrants 테이블을 기반으로 문서 및 워크스페이스(Workspace) 단위의 다대다(N:M) 공유 체계와 계층적 퍼미션(workspace.knowledge, sharing.public_knowledge 등)을 갖추고 있다. 사용자 및 그룹 제어에 있어서도 '전역 기본값(Global Defaults)'과 '그룹 재정의(Group Overrides)'가 결합된 가산적(Additive) 논리 아키텍처를 운영한다.[4] 반면 OpenMake LLM은 단순히 요금제 기반의 계층(Tool Tiers: free/pro/enterprise) 및 범위(Scope) 제어만을 제공하며 데이터 그룹화 및 리소스 권한 제어 모듈이 전무한 상태이다.

이를 보완하기 위해 OpenMake LLM 백엔드 아키텍처에 리소스(문서, 프롬프트, 모델 파이프라인)의 소유권(Ownership)과 공유 대상(User/Group)을 매핑하는 중앙 집중식 권한 제어 테이블을 도입해야 한다.[11] 시스템의 권한 논리는 "거부(Deny)" 권한을 명시적으로 두지 않고, "True가 False를 덮어쓴다(True Overrides False)"는 긍정적 승인 모델을 채택하는 것이 아키텍처 복잡도를 낮추는 데 효과적이다.[4] 이를 통해 기본적으로 모든 사용자에게는 최소 권한(Principle of Least Privilege)을 부여하고, '인사팀'이나 '재무연구원'과 같은 특정 그룹에만 기밀 파일 업로드 권한이나 부서 전용 지식 베이스 접근 권한을 선별적으로 덮어씌워 부여하는 유연한 관리가 가능해진다.[4] 또한 활성 채팅 중 로컬 문서 파싱 기능을 강제로 비활성화하고 브라우저 내부에서만 처리하도록 제한하는 '임시 채팅(Temporary Chat)' 강제 옵션을 도입하여 민감 정보의 영구 저장을 원천 차단하는 기능이 보완되어야 한다.[4]

더불어 웹 검색 파이프라인 통합 시 필연적으로 발생하는 서버 측 요청 위조(Server-Side Request Forgery, SSRF) 공격에 대한 방어 로직이 결여되어 있다. Open WebUI가 URLProcessingMixin을 통해 내부 IP 차단 및 SSL 검증을 수행하는 것과 유사하게, OpenMake LLM은 사용자가 외부 도구(Firecrawl 웹 검색 등)를 호출하거나 URL을 입력할 때 내부 망(127.0.0.1, 10.x.x.x, 192.168.x.x 등)으로의 라우팅을 명시적으로 차단하고 도메인 평판을 검증하는 커스텀 보안 미들웨어를 도입해야 한다.

[우선순위 1] RAG 생태계의 패러다임 전환: 단일 검색에서 하이브리드 및 다중 소스 융합 프레임워크로

검색 증강 생성(Retrieval-Augmented Generation, RAG) 기술은 환각(Hallucination) 현상을 제어하고 모델의 응답에 신뢰성을 부여하는 가장 핵심적인 파이프라인이다. 비교 명세서에 따르면, Open WebUI는 11종의 벡터 데이터베이스 어댑터를 지원하며 하이브리드 검색과 교차 인코더(Cross-encoder) 리랭킹 알고리즘을 결합한 고도화된 생태계를 구축하고 있다. 반면 OpenMake LLM은 2계층 시맨틱 캐시(Exact Match 및 L2 Cosine)와 4-Pillar 컨텍스트 엔지니어링 등 독자적인 응답 가속 기능은 우수하나, 정작 검색의 근간이 되는 리트리벌(Retrieval) 아키텍처는 단일 pgvector(768차원, 코사인 유사도)에만 의존하여 정밀도 하락이라는 본질적인 취약점을 안고 있다. 정보 검색의 품질을 극대화하기 위해 다음과 같은 아키텍처 대수술이 필요하다.

BM25 희소 배열과 밀집 벡터를 융합한 하이브리드 검색 엔진 구축

순수 임베딩에 의존하는 단일 밀집 벡터(Dense Vector) 검색 구조는 문맥적 의미를 파악하는 데는 탁월하지만, 사용자가 특정 '제품 일련번호', '에러 코드', 또는 고유명사를 검색할 경우 관련성을 제대로 포착하지 못하고 엉뚱한 문서를 반환하는 치명적인 한계가 있다.[12] 이를 극복하기 위해 어휘적 정밀도(Lexical Precision)를 담당하는 BM25(희소 검색) 알고리즘의 도입이 시급하다.[13] BM25는 질의에 포함된 단어의 문서 내 등장 빈도(TF)와 전체 문서 집합에서의 역문서 빈도(IDF)를 계산하여 정확히 일치하는 키워드의 가중치를 증폭시키는 메커니즘을 수행한다.[14]

OpenMake LLM은 현재 데이터베이스로 PostgreSQL을 사용하고 있으므로, 별도의 외부 검색 엔진(Elasticsearch 등)을 구축하여 동기화 지연을 유발하는 대신 데이터베이스 내부에서 하이브리드 검색을 직접 처리하는 아키텍처를 채택해야 한다.[13] 구체적으로 ParadeDB나 VectorChord와 같은 PostgreSQL 확장을 활용하여, 텍스트 토큰화를 통한 BM25 희소 배열 인덱스와 pgvector의 밀집 벡터 인덱스를 단일 테이블에서 동시에 관리하는 방식이 이상적이다.[12] 이를 통해 백엔드 애플리케이션인 Node.js 환경에서 단일 쿼리로 두 가지 검색 방식의 결과를 동시에 호출할 수 있어 네트워크 레이턴시를 최소화할 수 있다.

상호 순위 융합(RRF) 알고리즘 및 Cross-Encoder 기반 리랭킹 파이프라인

BM25 검색 결과와 벡터 시맨틱 검색 결과는 점수 분포 척도(Scale)가 완전히 다르기 때문에 단순한 수치 합산으로 우선순위를 매길 수 없다.[13] 하이브리드 검색의 효율성을 달성하기 위해서는 검색된 초기 문서 후보군을 적절히 결합하는 상호 순위 융합(Reciprocal Rank Fusion, RRF) 알고리즘의 도입이 필수적이다.[13] 백엔드 파이프라인은 두 검색 엔진에서 반환된 상위 K개의 후보군(예: 20개)을 취합하고, 각 문서가 랭크된 순위의 역수를 합산하여 새로운 융합 점수를 산출하는 중간 계층을 추가해야 한다.

초기 후보군이 융합된 이후에는 답변의 정확도를 극대화하기 위해 교차 인코더(Cross-encoder) 기반의 외부 리랭킹(Reranking) 파이프라인을 통과시켜야 한다.[15] 샴 네트워크 구조를 가지는 일반적인 임베딩 모델(Bi-encoder)과 달리, Cross-encoder는 질의와 문서를 동시에 모델의 입력으로 넣어 주의 집중(Attention) 메커니즘이 양방향으로 작용하게 함으로써 압도적으로 높은 관련성 예측 성능을 발휘한다. OpenMake LLM은 bge-reranker-v2-m3 등 다국어 처리 성능이 우수한 리랭킹 모델을 통합하고, UI에서 사용자가 최종 추출 후보 수(Top K Reranker)와

관련성 임계값(Relevance Threshold)을 동적으로 조절할 수 있는 구성 옵션을 제공해야 한다.[14]

다중 소스 통합 및 동적 컨텍스트 조절 아키텍처

기업의 지식은 내부 로컬 문서에만 고정되어 있지 않다. Open WebUI는 26종에 달하는 웹 검색 프로바이더(SearXNG, Brave Search, Perplexity 등)와 연동하여 최신 정보를 수집할 수 있으며, 유튜브 URL에서 직접 트랜스크립션을 추출하는 전용 RAG 파이프라인까지 갖추고 있다.[4] OpenMake LLM은 자체적인 심층 연구 파이프라인(Deep Research Service)을 보유하고 있으나 단일 외부 크롤링 서비스에 지나치게 의존적인 구조를 취하고 있다. 모델이 실시간으로 웹 브라우징을 수행하고 fetch_url 도구를 사용하여 전체 페이지 컨텍스트를 동적으로 분석할 수 있는 에이전틱(Agentic) 검색 플러그인 생태계로의 확장이 필요하다.[4]

또한, 업로드된 문서를 모델에 주입하는 방식에 있어서도 유연성이 요구된다. 현재 OpenMake LLM은 무조건적으로 문서를 잘게 쪼개는 방식에 의존하지만, 짧은 보고서나 코드 파일의 경우 단편화된 청크보다 문서 원본 전체를 프롬프트에 주입하는 것이 요약 정확도를 높인다. 따라서 사용자가 UI 상에서 RAG 청크 기반의 '집중 검색(Focused Retrieval)' 모드와 원문 전체를 컨텍스트로 주입하는 '완전 주입형(Full Injection)' 모드를 클릭 한 번으로 토글할 수 있는 인터페이스 및 백엔드 라우팅 로직이 보완되어야 한다.[4]

RAG 품질 및 신뢰성 검증을 위한 자동화된 평가 지표**(Metrics)** 내재화

아키텍처 관점에서 RAG 시스템의 위험성은 모델이 잘못된 정보를 생성하더라도 그것이 매끄럽고 유창한 문장으로 포장되어 '조용한 실패(Silent Failures)'를 초래한다는 데 있다.[17] 철저한 평가가 수반되지 않은 RAG 시스템은 기업 내 사용자들의 신뢰를 빠르게 상실하게 만든다.[17] Open WebUI를 넘어서기 위해, OpenMake LLM은 RAG 파이프라인 구축 단계를 넘어 그 품질을 정량적으로 평가하고 모니터링할 수 있는 RAG Triad(RAG 삼원조) 중심의 평가 프레임워크를 관리자 대시보드에 내장해야 한다.[18]

이 평가 아키텍처는 세 가지 핵심 축으로 구성된다. 첫째, 충실성**(Groundedness/Faithfulness)** 지표는 LLM 생성기가 검색된 문서의 사실(Fact) 범위를 벗어나 환각을 일으켰는지를 평가한다(참조 모델 기준 F1: 81%, 정밀도: 78%).[18] 둘째, 컨텍스트 관련성**(Context Relevance)** 지표는 시스템의 리트리버(Retriever) 모듈이 사용자의 질의에 얼마나 적합한 문서를 데이터베이스에서 끌어왔는지를 측정하여 임베딩 및 하이브리드 검색의 성능을 추적한다.[18] 셋째, 답변 관련성**(Answer Relevancy)** 지표는 최종적으로 도출된 응답이 원래 질문의 의도에 간결하게 부합하는지를 분석한다.[18] 추가적으로 모델이 RAG 파이프라인 내에서 쿼리 분해 로직 등 외부 도구를 호출할 때 적절한 인수를 생성했는지를 검증하는 도구 정확성**(Tool Correctness)** 평가 모듈을 통합함으로써 [19], 관리자가 임베딩 모델 교체나 파인튜닝 시 발생하는 품질의 변화를 즉각적으로 파악할 수 있는 완전한 피드백 루프를 완성해야 한다.

[우선순위 **2]** 지능형 문서 전처리 및 다중 모드**(Multi-modal)** 파이프라인 구축

기업 데이터의 대다수는 단순한 텍스트 파일이 아니라, 복잡한 서식과 표, 차트가 포함된 PDF,

DOCX, PPTX 등의 비정형 문서 형태로 존재한다. 기술 스택 비교에서 가장 치명적인 차이를 보이는 영역 중 하나가 바로 이 전처리(Preprocessing) 단계이다. Open WebUI는 Apache Tika부터 Docling, Mistral OCR에 이르는 7종 이상의 추출 엔진을 제공하며 다중 포맷을 지원하는 반면, OpenMake LLM은 기본 텍스트 추출 수준의 pdf-parse와 문자 인식률에 한계가 명확한 tesseract.js 엔진 단일 구조에 머물러 있어, 표나 차트가 포함된 보고서를 입력할 경우 정보의 구조적 붕괴가 불가피하다.[20]

추상 구문 트리(AST) 기반의 의미론적(Semantic) 문서 파싱 엔진 도입

단순 텍스트 기반으로 문서를 읽어 들여 1,000자 단위의 고정된 크기(CHUNK_SIZE)로 기계적인 분할(Recursive Character Chunking)을 수행하는 현재의 OpenMake LLM 방식은 문맥의 단절을 초래한다. 문단의 중간이 잘리거나, 다단으로 구성된 표 데이터가 무작위로 섞이게 되어 LLM이 컨텍스트를 잃게 되는 원인이 된다.

이를 근본적으로 해결하기 위해 Node.js 백엔드 생태계에 최적화된 officeparser 라이브러리(v6.0.0 이상)를 도입하여 문서를 단순 평문이 아닌 추상 구문 트리(Abstract Syntax Tree, AST) 구조로 파싱하는 아키텍처로 전면 개편해야 한다.[22] AST 파싱을 적용하면 문서 내부의 구조적 계층이 heading, paragraph, list, table 등의 노드 단위로 식별되며, 제목 노드(metadata.level === 1)를 기준으로 완결성 있는 단락을 묶어 시맨틱 청킹(Semantic Chunking)을 수행할 수 있게 된다.[22]

특히 LLM이 가장 취약점을 보이는 표(Table) 데이터의 경우, 파싱된 AST 트리 내의 table -> row -> cell 계층 구조를 순회하여 기계가 읽기 쉬운 형태의 CSV 문자열이나 마크다운 테이블 포맷으로 변환한 뒤 독립적인 메타데이터와 함께 벡터 데이터베이스에 색인하는 별도의 로직을 구성해야 한다.[22] 추가적으로 PPTX와 같은 프레젠테이션 파일 처리 시 기업 환경에서 핵심 정보가 담겨있는 '발표자 노트' 영역을 식별하여(putNotesAtLast 옵션 등) 문서 텍스트 후단에 누락 없이 병합하는 세밀한 파이프라인 구성이 뒷받침되어야 한다.[22] 이러한 AST 기반의 추출을 통해 DOCX, PPTX, XLSX 문서 포맷에 대한 지원 부족을 동시에 해결할 수 있다.

시각 언어 모델(VLM) 및 다단계 구조 인식 하이브리드 OCR 앙상블

재무제표, 설계 도면, 브로슈어 등 PDF 형태의 이미지 스캔본이나 복합 문서를 처리할 때, 단순한 텍스트 레이어 추출이나 레거시 OCR 모듈(tesseract.js)은 데이터의 구조적 정렬을 파괴하고 치명적인 오인식을 발생시킨다.[21] 오픈소스 생태계에서 단일 AI 모델이나 단일 OCR 도구만으로 완벽한 문서를 재구성하는 것은 불가능에 가깝다. 따라서 OpenMake LLM은 시각 지능(Vision)과 구조적 파싱을 결합한 다단계 하이브리드 폴백(Fallback) 파이프라인을 구축해야 한다.[23]

1. **구조-인식(Structure-aware) 전용 OCR 통과:** 문서 인식 파이프라인의 첫 단계에서는 tesseract 대신, 레이아웃 보존과 이미지 영역 식별에 특화된 Marker, GOT-OCR2.0, 또는 Docling과 같은 최신 엔진을 통해 텍스트 블록과 시각적 요소의 경계(Bounding Box)를 식별한다.[21] 이 단계에서 차트, 수식, 서명이 포함된 시각적 영역은 별도의 이미지 파일로 크롭(Crop)되고, 해당 위치 정보가 포함된 마크다운 뼈대 파일이 생성된다.[24]
2. **VLM(Vision Language Model)을 통한 의미론적 분석:** 잘려진 이미지 영역(표, 그래프, 인포그래픽)은 Llama 3.2 11B Vision Instruct, MiniCPM-V-2_6, 또는 복잡한 차트 해석에

특화된 ChartGemma와 같은 VLM 인퍼런스 엔진으로 병렬 전달된다.[21] VLM은 단순히 글자를 읽는 것을 넘어 시각적 요소가 의미하는 바를 서술형 데이터나 구조화된 JSON 데이터로 번역하는 역할을 수행한다.[21]

3. **LLM** 오케스트레이션을 통한 최종 융합 정제**:** 1단계에서 추출된 마크다운 텍스트와 2단계에서 VLM이 해석한 시각 데이터를 텍스트 전용 LLM(파서 역할)에 주입한다. 이 마지막 단계에서 LLM은 어긋난 정렬을 바로잡고, 문맥을 추론하여 끊어진 문장을 연결하며, 병합된 셀(Merged Cells) 구조를 논리적으로 복원하는 '슈퍼 정제기(Super-intelligent parser)' 역할을 수행하여 완벽한 데이터를 벡터 데이터베이스로 이관한다.[23]

이러한 모듈식 앙상블 접근 방식은 단일 도구의 한계를 상호 보완하며 복잡한 엔터프라이즈 문서 추출에 있어 절대적인 우위를 확보할 수 있게 한다.

[우선순위 **3]** 프론트엔드 아키텍처 혁신 및 확장 가능한 사용자 경험**(UX)** 극대화

플랫폼의 백엔드 파이프라인이 아무리 강력하더라도, 사용자가 직관적으로 정보를 시각화하고 워크플로우를 통제할 수 없다면 그 가치는 반감된다. Open WebUI는 SvelteKit과 TailwindCSS를 기반으로 TipTap 에디터, 수학 수식 렌더러(KaTeX), 마크다운 기반의 코드 렌더링을 융합하여 상용 서비스인 ChatGPT에 버금가는 UX를 제공하고 있다 [Image 8]. 반면 OpenMake LLM은 Vanilla JS 환경에서 기본 textarea와 단순 CSS 토큰만을 사용하여 아키텍처 다이아그램이나 동적인 시각 자료의 렌더링 능력이 치명적으로 결여되어 있다.

다이어그램 애즈 코드**(Diagrams as Code)** 아키텍처 도입**: Mermaid.js** 네이티브 렌더링

시스템 아키텍처, 데이터베이스 스키마, 복잡한 의사결정 트리를 설명할 때 텍스트만으로 전달하는 방식은 인지적 병목을 유발한다. LLM은 플로우차트를 논리적인 마크다운 코드 블록으로 생성하는 데 탁월한 능력을 갖추고 있으므로, 이를 프론트엔드에서 실시간 이미지로 변환해 주는 시각화 렌더러의 존재는 현대 AI 플랫폼의 필수 요소이다.[25]

Open WebUI의 네이티브 렌더링 기능을 벤치마킹하여, OpenMake LLM은 Vanilla JS 기반의 프론트엔드 구조에 Mermaid.js 라이브러리를 동적으로 통합하는 아키텍처를 도입해야 한다.[27] 기술적 구현은 ESM 모듈 스크립트를 통해 브라우저 측에 Mermaid.js 라이브러리(https://cdn.jsdelivr.net/npm/mermaid@11/dist/mermaid.esm.min.mjs)를 비동기식으로 가져오는(Import) 방식으로 진행된다.[27] 채팅 인터페이스 DOM에서 LLM의 응답 스트림이 완료된 직후, 브라우저가 정규표현식을 통해 마크다운 코드 블록 중 mermaid 클래스 태그가 포함된 텍스트 블록(graph LR, sequenceDiagram 등)을 실시간으로 감지해야 한다.[27] 이후 백그라운드에서 mermaid.render() API를 호출하여 해당 텍스트 코드를 동적 SVG 이미지로 변환하여 화면에 주입(Inject)하는 인터랙티브 모듈이 추가되어야 한다.[28] 사용자가 렌더링된 다이어그램 위에 마우스를 올릴 때 동적인 툴팁이 표시되거나, 클릭 한 번으로 소스 코드를 복사하여 사내 위키(Confluence, Notion 등) 시스템에 재사용할 수 있도록 클립보드 내보내기

유틸리티를 결합하여 사용자 생산성을 극대화해야 한다.[25] 더 나아가 수식 표현을 위한 KaTeX 라이브러리도 동일한 메커니즘으로 연동하여 데이터 과학 및 엔지니어링 도메인에서의 활용성을 끌어올려야 한다.[30]

리치 텍스트 에디터 및 향상된 사용자 상호작용 설계

OpenMake LLM의 입력 인터페이스는 일반적인 텍스트 영역(textarea)에 국한되어 있어, 사용자가 프롬프트를 정교하게 서식화하거나 긴 텍스트와 코드 블록을 조합하여 입력하는 데 큰 제약이 따른다. 이를 해결하기 위해 TipTap이나 CodeMirror와 같은 모듈형 리치 텍스트 에디터를 프론트엔드에 포팅해야 한다. 구문 강조(Syntax Highlighting) 로직 또한 highlight.js 단일 구성에서 나아가 대화형 편집이 가능한 자동 완성 및 실시간 린팅(Linting) 기능을 포함하는 구조로 고도화되어야 한다. 반응형 모바일 환경 지원을 넘어 프로그레시브 웹 앱(PWA) 서비스 워커를 개선하고, ARIA 라벨을 포함한 키보드 내비게이션 접근성(A11y) 표준을 충족하도록 CSS 토큰 및 DOM 구조를 재설계해야 한다 [Image 8]. 아울러 글로벌 엔터프라이즈 환경에 맞게 런타임 언어 감지에 의존하는 기존의 한국어/영어 로케일 구조를 벗어나, 다국어 i18n 번역 파일 구조를 독립시켜 커뮤니티 기여를 통해 수십 개의 언어로 확장 가능한 뼈대를 갖추어야 한다.

[우선순위 3] 데이터 거버넌스, 지식 아키텍처 및 시스템 관측성(Observability) 확보

최종적으로, 기업용 AI 생태계가 안정적으로 동작하기 위해서는 데이터의 물리적 저장 구조 최적화와 더불어, 시스템의 리소스 사용을 투명하게 추적할 수 있는 모니터링 체계가 요구된다.

계층적 지식 베이스 아키텍처 및 커뮤니티 확장성

현재 OpenMake LLM의 지식 베이스 아키텍처는 단일 파일 단위의 단순 매핑 구조에 머물러 있어 대규모 문서 처리에 부적합하다. Open WebUI와 같이 Knowledge -> KnowledgeFile -> File로 이어지는 다대다(N:M) 구조의 관계형 매핑 체계로 데이터베이스 스키마를 마이그레이션해야 한다 [Image 4]. 이를 통해 여러 부서의 파일들을 커스텀 벡터 컬렉션 단위로 논리적으로 그룹화하고, 하나의 쿼리로 다중 디렉터리를 동시에 횡단 검색할 수 있는 기능을 제공해야 한다. 또한 파일 삭제 시 연결된 벡터 데이터베이스 내의 시맨틱 임베딩 및 메타데이터 찌꺼기를 트리거(Trigger)를 통해 자동으로 정리(Cleanup)하는 무결성 유지 로직이 추가되어야 한다.[4]

생태계적 관점에서, Open WebUI는 마켓플레이스 및 프롬프트 커뮤니티를 통해 도구와 파이프라인의 공유가 활발히 이루어진다. OpenMake LLM은 강력한 자율 에이전트 모듈(22개 동적 프로필)을 보유하고 있음에도 폐쇄적인 구조로 인해 사용자 간의 유스케이스 공유가 차단되어 있다. 중앙 서버 또는 사내 레지스트리와 연동하여 각 사용자가 생성한 에이전트 페르소나, 스킬 스크립트(skill-manager.ts), 그리고 산업 전문 템플릿을 손쉽게 Import/Export 하고 공유할 수 있는 플러그인 레지스트리 아키텍처를 신설해야 한다.

엔터프라이즈 모니터링 대시보드 및 시스템 관측성(Observability) 구축

여러 테넌트와 모델이 복합적으로 작동하는 환경에서 백엔드 리소스 모니터링은 필수적이다.

Open WebUI의 관리자 대시보드 벤치마킹을 바탕으로, OpenMake LLM은 사용자별, 부서별, 그리고 연결된 LLM 별 토큰 소비량, 트래픽 점유율(Share/Percentage), 쿼리 처리 지연율을 한눈에 파악할 수 있는 고급 애널리틱스 대시보드를 내장해야 한다.[2]

더 나아가 분산 컴퓨팅 환경에서의 추적성을 확보하기 위해 OpenTelemetry 규격을 준수하는 트레이스(Trace), 로그, 메트릭 방출(Export) 인터페이스를 구축하여, 기업에서 이미 운영 중인 Prometheus, Grafana 등의 관측성(Observability) 스택과 매끄럽게 연동되도록 시스템을 확장해야만 상용 프로덕션 수준의 가용성을 인정받을 수 있다.[2]

결론: "깊은 파이프라인"과 "범용 연결성"의 하이브리드 진화 전략

분석된 내용을 종합하면, Open WebUI는 광범위한 호환성, 엔터프라이즈 보안 생태계, 그리고 하이브리드 RAG 검색 아키텍처를 바탕으로 현존하는 가장 보편적이고 완성도 높은 플랫폼의 형태를 띠고 있다. 반면 OpenMake LLM은 '단일 백엔드와 깊은 파이프라인' 철학 아래 A2A 토론 엔진, 시맨틱 캐시 라우팅, 심층 연구 서비스 등 복합 추론과 자율 에이전트 영역에서 Open WebUI를 능가하는 독창적인 경쟁력을 입증하였다.

따라서 OpenMake LLM이 나아가야 할 차세대 진화 방향은 본질적인 장점인 에이전틱 파이프라인의 깊이를 유지하면서, 결여되어 있는 엔터프라이즈 기반 인프라를 수용하는 '하이브리드 전략'을 취하는 것이다. 즉, 1순위로 SCIM 2.0 및 LDAP을 통한 계정 프로비저닝, 가산적 RBAC 보안 인프라, 그리고 BM25와 pgvector를 융합한 RRF 기반 하이브리드 정보 검색 엔진을 최우선으로 이식하여 기업 채택의 허들을 허물어야 한다. 이후 2순위로 AST 기반의 의미론적 청킹과 VLM 기반 폴백 다중 모드 OCR 엔진을 결합하여 데이터 유입의 신뢰성을 극대화하고, 최종적으로 Mermaid.js 동적 렌더링 및 모니터링 시스템 등 UX 확장성을 완성해 나간다면, 향후 대규모 엔터프라이즈 시장에서 강력한 자율형 보안 AI 생태계를 주도하는 핵심 아키텍처로 자리 잡을 수 있을 것이다.

참고 자료

1. Open WebUI: Home, 3월 2, 2026에 액세스, https://docs.openwebui.com/
2. Roadmap | Open WebUI, 3월 2, 2026에 액세스, https://docs.openwebui.com/roadmap/
3. SCIM 2.0 | Open WebUI, 3월 2, 2026에 액세스, https://docs.openwebui.com/features/access-security/auth/scim/
4. Permissions | Open WebUI, 3월 2, 2026에 액세스, https://docs.openwebui.com/features/access-security/rbac/permissions/
5. How to build a SCIM 2.0 endpoint: Developer's guide | Scalekit Blog, 3월 2, 2026에 액세스, https://www.scalekit.com/blog/build-scim-endpoint
6. Implementing a SCIM API for Your Application: A Comprehensive Guide | SSOJet - Enterprise SSO & Identity Solutions, 3월 2, 2026에 액세스, https://ssojet.com/blog/implementing-a-scim-api-for-your-application-a-comprehensive-guide

7. Notes on building a SCIM endpoint from scratch - DEV Community, 3월 2, 2026에 액세스, https://dev.to/saif_shines/notes-on-building-a-scim-endpoint-from-scratch-cgc
8. Use LDAP and Active Directory to authenticate Node.js users - IBM Developer, 3월 2, 2026에 액세스, https://developer.ibm.com/tutorials/se-use-ldap-authentication-authorization-nodejs-cloud-application/
9. implement simple Nodejs server and LDAP | by Arashramy - Medium, 3월 2, 2026에 액세스, https://medium.com/@arashramy/implement-simple-nodejs-server-and-ldap-2756b6344f12
10. node.js - Active Directory authentication with NodeJS - Stack Overflow, 3월 2, 2026에 액세스, https://stackoverflow.com/questions/50622797/active-directory-authentication-with-nodejs
11. feat: User Permission Toggle for RAG Document Access · Issue #1096 - GitHub, 3월 2, 2026에 액세스, https://github.com/open-webui/open-webui/issues/1096
12. I implemented Hybrid Search (BM25 + pgvector) in Postgres to fix RAG retrieval for exact keywords. Here is the logic. - Reddit, 3월 2, 2026에 액세스, https://www.reddit.com/r/Rag/comments/1pcvtan/i_implemented_hybrid_search_bm25_pgvector_in/
13. Hybrid Search in PostgreSQL: The Missing Manual | ParadeDB, 3월 2, 2026에 액세스, https://www.paradedb.com/blog/hybrid-search-in-postgresql-the-missing-manual
14. Hybrid search with Postgres Native BM25 and VectorChord, 3월 2, 2026에 액세스, https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html
15. Features | Open WebUI, 3월 2, 2026에 액세스, https://docs.openwebui.com/features/
16. Multi-Source RAG with Hybrid Search and Re-ranking in ... - Medium, 3월 2, 2026에 액세스, https://medium.com/@richard.meyer596/multi-source-rag-with-hybrid-search-and-re-ranking-in-openwebui-8762f1bdc2c6
17. RAG systems: Best practices to master evaluation for accurate and reliable AI., 3월 2, 2026에 액세스, https://cloud.google.com/blog/products/ai-machine-learning/optimizing-rag-retrieval
18. Benchmarking LLM-as-a-Judge for the RAG Triad Metrics - Snowflake, 3월 2, 2026에 액세스, https://www.snowflake.com/en/engineering-blog/benchmarking-LLM-as-a-judge-RAG-triad-metrics/
19. LLM Evaluation Metrics: The Ultimate LLM Evaluation Guide - Confident AI, 3월 2, 2026에 액세스, https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation

20. Best Open-Source OCR Tools in 2025: A Comparison - Unstract, 3월 2, 2026에 액세스, https://unstract.com/blog/best-opensource-ocr-tools-in-2025/
21. Approaches to PDF Data Extraction for Information Retrieval | NVIDIA Technical Blog, 3월 2, 2026에 액세스, https://developer.nvidia.com/blog/approaches-to-pdf-data-extraction-for-information-retrieval/
22. officeparser - npm, 3월 2, 2026에 액세스, https://www.npmjs.com/package/officeparser
23. Seeking Advice: Reliable OCR/AI Pipeline for Extracting Complex Tables from Reports, 3월 2, 2026에 액세스, https://discuss.huggingface.co/t/seeking-advice-reliable-ocr-ai-pipeline-for-extracting-complex-tables-from-reports/167296
24. Integrating good OCR and Vision models into something that can dynamically aid in document research with a LLM : r/LocalLLaMA - Reddit, 3월 2, 2026에 액세스, https://www.reddit.com/r/LocalLLaMA/comments/1g3h1s8/integrating_good_ocr_and_vision_models_into/
25. Mermaid.js: The Code-First Approach to Technical Diagrams | by Vaij Bharamshetty, 3월 2, 2026에 액세스, https://medium.com/@vaijrb/mermaid-js-the-code-first-approach-to-technical-diagrams-6a3c4247d842
26. Using Mermaid JS for Insanely Quick Diagramming - Will Kennard, 3월 2, 2026에 액세스, https://www.willkennard.com/digital-blog/using-mermaid-js-for-insanely-quick-diagramming
27. Usage - Mermaid, 3월 2, 2026에 액세스, https://mermaid.ai/open-source/config/usage.html
28. How to render a mermaid flowchart dynamically? - Stack Overflow, 3월 2, 2026에 액세스, https://stackoverflow.com/questions/65212332/how-to-render-a-mermaid-flowchart-dynamically
29. Mermaid.js: A Complete Guide - Swimm, 3월 2, 2026에 액세스, https://swimm.io/learn/mermaid-js/mermaid-js-a-complete-guide
30. Adding LaTeX math support via Katex · Issue #2776 · mermaid-js/mermaid - GitHub, 3월 2, 2026에 액세스, https://github.com/mermaid-js/mermaid/issues/2776
31. Releases · open-webui/open-webui - GitHub, 3월 2, 2026에 액세스, https://github.com/open-webui/open-webui/releases